

From User to Contributor

Fixing Bugs and Adding Features in Clang

Matheus Izvekov
C++ Alliance — mizvekov@gmail.com

CppCon 2026

1. Why Contribute?

Motivations you already have

- A **bug** that affects your own codebase: you filed the issue, but nobody picked it up.
- A **diagnostic** you wish existed: a class of mistake your team keeps making in review.
- A **standard conformance gap**: code that should be ill-formed and isn't, or the reverse.

The biggest mistake that prevents people from trying is the belief that you need to understand the whole compiler before touching it.

The LLVM contribution process

```
GitHub issue  →  local repro  →  patch  →  code review  →  merge
```

- Everything lives on GitHub, in a single repo: `llvm/llvm-project`.
- No separate bug tracker, no separate review tool, no mailing-list involved.
- Review happens directly on the pull request, by maintainers who own that area of code.

2. Setting Up a Development Build

Clone and configure

```
git clone https://github.com/llvm/llvm-project.git
cd llvm-project

cmake -G Ninja -S llvm -B build \
  -DLLVM_ENABLE_PROJECTS=clang \
  -DCMAKE_BUILD_TYPE=Release \
  -DLLVM_ENABLE_ASSERTIONS=ON
```

- `-S llvm -B build`: the source root is the `llvm/` subdirectory; everything else (`clang`, `lld`, …) is a sibling project enabled via `LLVM_ENABLE_PROJECTS`.
- Only enable what you need: configuring `clang` alone keeps the build smaller.
- Keep assertions **on**, even in a Release build: they help catch problems early.

`ccache`: don't rebuild what you didn't change

```
cmake ... -DLLVM_CCACHE_BUILD=ON
```

- On LLVM, ccache is mostly configured through project specific CMake variables.
- This is very helpful to allow you to change branches and back without recompiling everything.

Build only what you're testing

```
ninja -C build clang
```

- Not `ninja -C build all`: that also builds every other tool.
- First full build of `clang` alone: expect anywhere from 20 minutes to well over an hour, depending on hardware and parallelism.

Sanity check: run the test suite

```
ninja -C build check-clang
```

- `check-clang` runs Clang's own regression tests (`clang/test/**`) through `lit`, Clang's test runner.
- Tens of thousands of tests, a few minutes to run on a modern machine.
- If this isn't fully green on a fresh clone, most likely something's wrong with the build, not with Clang.
  But it's possible you have checked out a commit which is broken upstream.
  When the latter happens, some other contributor will most likely revert the offending commit soon enough.

3. Tour of the Codebase

The pipeline

```
source text
   |  Lex     (clang/lib/Lex)
   v
tokens
   |  Parse   (clang/lib/Parse)
   v
parse actions
   |  Sema    (clang/lib/Sema)
   v
AST           (clang/lib/AST)
   |  CodeGen (clang/lib/CodeGen)
   v
LLVM IR  ->  backend  ->  object code
```

Sema is where almost every diagnostic and almost every rule of the language lives.
Parsing and Sema are interleaved, not strictly sequential: the parser calls into Sema as it goes.

Key directories

Path	What lives there
`clang/lib/Lex`	Tokenizer, preprocessor
`clang/lib/Parse`	Recursive-descent parser
`clang/lib/Sema`	Type checking, overload resolution, template instantiation, most diagnostics
`clang/lib/AST`	AST node definitions, printers
`clang/include/clang/Basic/Diagnostic*.td`	Diagnostic text
`clang/test/`	Regression tests (`lit` + `FileCheck`, gtest unit tests)

Both of today's examples touch only two of these.

From an error message back to source

1. **Copy the exact diagnostic text** and `grep` for it in the `.td` files:

   ```
   grep -rn "invalid application of" clang/include/clang/Basic/Diagnostic*.td
   ```

   Some times this will not work straight away. For example because the strings can be split, or because some parts of the sentence can be one choice among a selector.

2. That gives you a diagnostic ID (`diag::err_foo`): then `grep -rn diag::err_foo clang/lib` finds every place that can actually emit it.

3. Use `git blame` around code which implements the diagnostic.
   Try to use that to get to the commit which introduced it.
   This is a huge help, as it will provide a link to all of the code involved and all of the tests.

4. Use **`clangd`** to jump to definitions and references once you're inside `clang/lib/Sema`.
   You would usually do this through some IDE integration with `clangd`.

4. Part 1: A new diagnostic

The warning: arithmetic on a boolean operand

```
bool f();

void g() {
  int x = f() + 1;   // legal... and almost always a mistake
}
```

- `bool` implicitly converts to `int`.
- Clang **already** warns about a few suspicious boolean idioms: bitwise negation of a `bool`, mixing `&&`/`||` with bitwise operators.
- It does **not** warn for plain `+ - * / %` on a `bool` operand.

Step 1: find the existing hook

We will start by looking for existing similar functionality in clang.

Looking at `clang/include/clang/Basic/DiagnosticSemaKinds.td`, and searching for other diagnostics involving `bool`, we come across something pretty close: `warn_bitwise_negation_bool`.

This is referenced by the `AnalyzeImplicitConversions` function in `clang/lib/Sema/SemaChecking.cpp`:

```
if (const auto *UO = dyn_cast<UnaryOperator>(SourceExpr))
  if (UO->getOpcode() == UO_Not &&
      UO->getSubExpr()->isKnownToHaveBooleanValue())
    S.Diag(UO->getBeginLoc(), diag::warn_bitwise_negation_bool)
        << OrigE->getSourceRange() << T->isBooleanType()
        << FixItHint::CreateReplacement(UO->getBeginLoc(), "!");
```

This function walks every implicit conversion Sema performs, looking for suspicious source expressions.

Step 2: declare the new diagnostic

We add a new entry into `clang/include/clang/Basic/DiagnosticSemaKinds.td`:

```
def warn_arithmetic_bool_operand : Warning<
  "arithmetic operation on a boolean operand">,
  InGroup<BoolOperation>, DefaultIgnore;
```

- `InGroup<BoolOperation>` makes the diagnostic be controlled by `-Wbool-operation`.
- `DefaultIgnore` means this diagnostic must be enabled explicitly: Even `-Wall` won't enable it, the user must pass `-Wbool-operation` or `-Weverthing`.

Step 3: extend the unary case (`+x`, `-x`)

```
-  if (const auto *UO = dyn_cast<UnaryOperator>(SourceExpr))
-    if (UO->getOpcode() == UO_Not &&
-        UO->getSubExpr()->isKnownToHaveBooleanValue())
+  if (const auto *UO = dyn_cast<UnaryOperator>(SourceExpr);
+      UO && UO->getSubExpr()->isKnownToHaveBooleanValue()) {
+    if (UO->getOpcode() == UO_Not) {
       S.Diag(UO->getBeginLoc(), diag::warn_bitwise_negation_bool)
           << OrigE->getSourceRange() << T->isBooleanType()
           << FixItHint::CreateReplacement(UO->getBeginLoc(), "!");
+    } else if (UO->getOpcode() == UO_Plus || UO->getOpcode() == UO_Minus) {
+      S.Diag(UO->getBeginLoc(), diag::warn_arithmetic_bool_operand)
+          << OrigE->getSourceRange() << T->isBooleanType()
+          << FixItHint::CreateInsertion(UO->getSubExpr()->getBeginLoc(), "(int)");
+    }
+  }
```

Widen the existing `if` to match `bool` operands generally, then branch on the specific opcode.

The `FixItHint` suggests the fix: insert `(int)` before the operand.

Step 4: extend the binary case (`x + y`, `x * y`, ...)

```
   if (auto *BO = dyn_cast<BinaryOperator>(SourceExpr)) {
     if ((BO->getOpcode() == BO_And || BO->getOpcode() == BO_Or) && ...) {
       ...
       return;
+    } else if (BO->getOpcode() == BO_Add || BO->getOpcode() == BO_Sub ||
+               BO->getOpcode() == BO_Mul || BO->getOpcode() == BO_Div ||
+               BO->getOpcode() == BO_Rem) {
+      for (const Stmt *Stmt : BO->children()) {
+        const auto *E = cast<Expr>(Stmt);
+        if (E->isKnownToHaveBooleanValue())
+          S.Diag(E->getBeginLoc(), diag::warn_arithmetic_bool_operand)
+              << E->getSourceRange()
+              << FixItHint::CreateInsertion(E->getBeginLoc(), "(int)");
+      }
     }
   }
```

Walking `BO->children()` instead of hand-checking `getLHS()` / `getRHS()` separately means **either** operand being `bool` is caught, and the fix-it lands on whichever operand is actually boolean, including both.

Step 5: write the test

We can figure out how to write this test by looking for existing tests for `-Wbool-operation`, and adapting as necessary.

Having done that, we can create a new test file `clang/test/SemaCXX/warn-bool-arithmetic.cpp`:

```
// RUN: %clang_cc1 -fsyntax-only -verify -Wbool-operation %s
// RUN: %clang_cc1 -fsyntax-only -Wbool-operation \
// RUN:   -fdiagnostics-parseable-fixits %s 2>&1 | FileCheck %s

bool f();

static void g() {
  (void)+f();
  // expected-warning@-1 {{arithmetic operation on a boolean operand}}
  // CHECK: fix-it:"{{.*}}":{[[@LINE-2]]:10-[[@LINE-2]]:10}:"(int)"

  (void)(f() % 1);
  // expected-warning@-1 {{arithmetic operation on a boolean operand}}
  // CHECK: fix-it:"{{.*}}":{[[@LINE-2]]:10-[[@LINE-2]]:10}:"(int)"
}
```

What's missing to make a pull-request from this

- More comprehensive test cases: Test all operators, test all operands.
- What's the value of this check to the users:
  - Is it catching mostly real bugs, or are there a lot of false positives?
  - Test it on real world code.
- It may be a good idea to create a separate group for this diagnostic, instead of reusing `-Wbool-operation`, if it's domain of applicability and quality differs significantly from the existing warnings.
- Support fix-it hint using `static_cast` instead of c-style casts, when available.

5. Part 2: Issue Walkthrough

The issue: a missed diagnostic

```
template <class T> struct A {
  int Array[sizeof(A)];   // A is incomplete here
};
```

- Inside its own definition, `A<T>` is **incomplete**: `sizeof` on it must be ill-formed, for every possible `T`.
- Clang accepts this silently today. The error will only surface when the class template is instantiated.

Step 1: write the test *before* the fix

Create a new test file `clang/test/SemaTemplate/incomplete-type.cpp`:

```
// RUN: %clang_cc1 -fsyntax-only -std=c++26 -verify %s

namespace IncompleteArrayInjected {
  template <class> struct A {           // #IncompleteArrayInjected-A
    int Array1[sizeof(A)];
    // expected-error@-1 {{invalid application of 'sizeof' to an incomplete type 'A<type-parameter-0-0>'}}
    // expected-note@#IncompleteArrayInjected-A {{definition of ... is not complete until the closing '}'}}
  };
}

namespace IncompleteArrayMemberClass {
  template <class> struct A {
    struct B;                           // #IncompleteArrayMemberClass-B
    int Array1[sizeof(B)];
    // expected-error@-1 {{invalid application of 'sizeof' to an incomplete type 'B'}}
  };
}
```

Written against **current** behavior: right now both compile with zero diagnostics.
This test currently fails, but that's the point.

Step 2: test just this one file

```
build/bin/llvm-lit -v clang/test/SemaTemplate/incomplete-type.cpp
```

```
FAIL: Clang :: SemaTemplate/incomplete-type.cpp
...
error: 'error' diagnostics expected but not seen:
  File ... Line XX: invalid application of 'sizeof' to an incomplete type ...
```

`llvm-lit` runs the `RUN:` lines in the file and checks them against the `expected-*` comments: no need to rebuild or rerun all of `check-clang` while iterating on one file.

Step 3: figure out why the check is missing

Clang parses and checks the validity of class templates just the same as it checks normal and instantiated classes.

But some times checks are skipped because they involve unknowns: we can't be certain a dependent type will conform to some restrictions.

So here is a plan for how to find where we need to change:

- Start with the initial test case, but append a line instantiating the class template.

```
template <class T> struct A {
  int Array[sizeof(A)];
};
template struct A<int>;
```

- Compile this test case and take notice of the diagnostic produced.

```
error: invalid application of 'sizeof' to an incomplete type 'A<int>'
```

- Figure out where this diagnostic is produced, as we learned how to previously, and set a breakpoint on it using your favorite debugger.

```
def err_sizeof_alignof_incomplete_or_sizeless_type : Error<
  "invalid application of '%0' to %select{an incomplete|sizeless}1 type %2">;
```

That's produced in the `CheckUnaryExprOrTypeTraitOperand` function in `clang/lib/Sema/SemaExpr.cpp`:

```
  if (RequireCompleteSizedType(
          OpLoc, ExprType, diag::err_sizeof_alignof_incomplete_or_sizeless_type,
          KWName, ExprRange))
    return true;
```

- Run the compiler again under the debugger, until the breakpoint is caught.
- Walk up the call stack until we find where the check is skipped when parsing the template.

We are going to find that right at the beginning of `CheckUnaryExprOrTypeTraitOperand`, everything is skipped when the expression has a dependent type.

This is going to be our first change:

```
 bool Sema::CheckUnaryExprOrTypeTraitOperand(QualType ExprType, ...) {
-  if (ExprType->isDependentType())
-    return false;
-
   // C++ [expr.sizeof]p2: ...
```

But redundantly, this is also skipped by the caller:

```
   QualType T = TInfo->getType();
-  if (!T->isDependentType() &&
-      CheckUnaryExprOrTypeTraitOperand(T, OpLoc, R, ExprKind, ...))
+  if (CheckUnaryExprOrTypeTraitOperand(T, OpLoc, R, ExprKind, ...))
     return ExprError();
```

Step 5: rerun tests

We can first check just the new test file, and verify we did fix it:

```
build/bin/llvm-lit -v clang/test/SemaTemplate/incomplete-type.cpp
```

```
PASS: Clang :: SemaTemplate/incomplete-type.cpp
```

But also, rerun all the tests:

```
ninja -C build check-clang
```

```
FAIL: Clang :: SemaCXX/builtin-assume-aligned-tmpl.cpp
FAIL: Clang :: <a ptrauth-related Sema test>
```

Ops, this fix broke ptrauth-related test cases.

Step 6: fixing the regression

The function `checkPtrAuthTypeDiscriminatorOperandType` (still in `SemaExpr.cpp`) never expected to see a dependent type either.

It now needs its own explicit guard, since it no longer gets one for free:

```
-  if (!T->isFunctionType() && !T->isFunctionPointerType() &&
-      !T->isFunctionReferenceType() && !T->isMemberFunctionPointerType()) {
+  if (!T->isDependentType() && !T->isFunctionType() &&
+      !T->isFunctionPointerType() && !T->isFunctionReferenceType() &&
+      !T->isMemberFunctionPointerType()) {
     S.Diag(Loc, diag::err_ptrauth_type_disc_undiscriminated) << T << ArgRange;
     return true;
   }
```

Step 7: rerun the tests one last time

```
ninja -C build check-clang
```

All tests pass.

This must mean our fix must be correct, right?

Let's try a new test case:

```
template <class> struct A {
  struct B {
    int Array[sizeof(A)];
  };
};
```

With our fix, clang now rejects this.

But if we try to instantiate that class template with the original clang, there is no error here.

So our change broke apparently valid code.

```
template <class> struct A {
  struct B {
    int Array[sizeof(A)];
  };
};
template struct A<int>; // OK
```

```
struct A {
  struct B {
    int Array[sizeof(A)]; // Not OK: A is incomplete here.
  };
};
```

There is something interesting going on here.

It seems templated classes have superpowers non-templated ones don't.

6. The Review Process

Opening the pull request

It didn't use to be this simple, but now this is the same as any other project on GitHub:

```
git checkout -b fix-bool-arithmetic-warning
git commit -a
git push -u my-llvm-fork add-bool-arithmetic-warning
```

- One logical change per PR.
- Push to your LLVM fork, unless you plan to push several stacked PRs, but this requires special attention to the branch name.
- Follow the link received from the push command in order to create the PR.

What reviewers actually look for

- **Test coverage**: Does the test fail without the fix, and pass with it?
- **Commit message**: `[clang] Short imperative summary`, body explains *why*, references the GitHub issue (`Fixes #12345`), if applicable.
- **Release note**: a `clang/docs/ReleaseNotes.md` entry for anything user-visible.
- **Code style**: There is an existing documentation on the LLVM coding style. Unless this adds a substantial amount of code, just match the surrounding code's style.

Common first-round feedback

- *"Please add a test for <related edge case>"*: reviewers who know the area will spot this.
- *"Can this diagnostic be worded more precisely?"*: diagnostic text gets scrutiny.
- *"Any additional tests performed?"*: Pre-commit CI will run the full test suite, but new warnings and changes which cause code to be ill-formed deserve greater scrutiny.
- Style nits are mostly taken care of by `git clang-format`, which is checked in CI.

Finding a reviewer

- Use **GitHub's suggested reviewers** for starters.
- `git blame -- <file>`: who wrote or last touched this and the surrounding code.
- `clang/Maintainers.md` lists code owners.
- The **`#clang` channel on the LLVM Discord**: ask "who reviews changes in this area" if you're stuck.

7. Where to Go Next

Finding your next bug

- The `good first issue` label on `llvm/llvm-project`.
- Filter open issues by the `clang:frontend` label for Sema bugs.

```
gh issue list --repo llvm/llvm-project --label "good first issue" --label "clang:frontend"
```

Join the community

- `#clang` **on the LLVM Discord**: day-to-day discussions, fastest turnaround.
- **LLVM Discourse forum** (`discourse.llvm.org`): longer-form design discussion, RFCs.
- **The LLVM Developers' Meeting**: in person conference.

Q&A

Slides, both patches: link will be shared after the session.

Matheus Izvekov — mizvekov@gmail.com